# GUIDE-2：读懂 sort.c

这份指南写给几乎没有学过操作系统的读者，目标是让你能逐行看懂 Lab02 Part II 的 `sort.c`：一个用多线程给整数列表排序的程序。实验要求见 `Lab02.md`。

**阅读前提**：会一点 C 语言（变量、`if` / `for` / `while`、函数、数组）。指针、`realloc`、`k++` 等较难的写法，会在用到的地方单独解释。

**和 GUIDE-1 的关系**：`sort.c` 和 Part I 的 `sudoku.c` 用的是同一套线程写法。这份指南可以单独阅读，第 2 节会简要回顾需要的操作系统知识，更详细的解释见 `GUIDE-1.md` 第 2 节。如果读过 GUIDE-1，可以重点看 2.4、2.5 和第 4 节，那里是 Part II 的新内容。

**阅读建议**：把 `sort.c` 打开放在旁边对照着看。第 1、2 节打基础，第 3 节看全貌，第 4 节逐段读代码，第 5 节动手运行，第 6 节解答常见疑问。

## 1. 程序要解决什么问题

`sort.c` 从标准输入读入一串整数（个数不限），把它们按从小到大的顺序排好，然后打印出来。

和 Part I 一样，这个实验的重点不是排序本身，而是**用多线程完成排序**。实验文档规定的做法是：

1. 把列表分成大小相等的两半；
2. 两个**排序线程**各自把一半排好序，排序算法自选（本程序用插入排序）；
3. 第三个**合并线程**把排好的两半合并成一个完整的有序列表，写进第二个数组；
4. 所有线程结束后，由父线程输出结果。

| 线程编号 | 角色 | 负责的工作 |
| --- | --- | --- |
| 0 | 排序线程 | 把列表的前一半排好序 |
| 1 | 排序线程 | 把列表的后一半排好序 |
| 2 | 合并线程 | 把排好的两半合并成一个有序列表 |

以实验文档里的列表为例（图中的 worker 0、1、2 就是线程 0、1、2）：

```
original list         7 12 19 3 18 4 2 6 15 8
                        /          \
two halves          7 12 19 3 18      4 2 6 15 8
                      |                 |
sort (worker 0, 1) 3 7 12 18 19       2 4 6 8 15
                        \          /
merge (worker 2)      2 3 4 6 7 8 12 15 18 19
```

为什么"分开排、再合并"能得到正确结果？因为两个各自有序的列表，很容易合并成一个有序列表（4.8 节会一步步演示）。这正是**归并排序**的基本思想。

## 2. 看懂代码需要的操作系统知识

这一节只讲读懂 `sort.c` 必需的概念。2.1～2.3 是对 GUIDE-1 第 2 节的简要回顾，2.4、2.5 是 Part II 新遇到的问题。

### 2.1 进程与线程

- **进程**：程序运行起来之后的实体。在终端输入 `./build/sort` 时，操作系统会创建一个进程：给它分配内存，把程序装进去，然后开始执行。
- **线程**：进程里真正在执行代码的"执行流"。每个进程启动时只有一个**主线程**，它从 `main()` 开始执行。主线程可以再创建新的线程（本程序称为**工作线程**），每个新线程去执行一个指定的函数。实验文档里说的**父线程**（parent thread），指的就是创建这些工作线程的主线程。

多个线程是**同时**运行的：在多核 CPU 上，它们可以在不同的核上真正并行；即使只有一个核，操作系统也会让它们快速轮流执行。**哪个线程先跑、每次跑多久，由操作系统决定，程序员无法预知。**

### 2.2 线程之间共享什么

同一个进程里的所有线程：

- **共享**：全局变量（定义在所有函数外面的变量）、用 `malloc` 或 `realloc` 申请的内存、程序代码。
- **各自私有**：函数里的局部变量。每个线程都有自己的一块内存区域（叫**栈**）来存放局部变量。

在 `sort.c` 中，全局变量 `list`、`sorted`、`count`，以及 `list`、`sorted` 指向的内存，所有线程都能访问。而线程 0 和线程 1 虽然同时执行同一个函数 `sorter`，它们各自的局部变量（`i`、`j`、`value` 等）互不干扰。

## 2.3 创建和等待线程：pthread

Linux 上创建和管理线程用 **pthread**（POSIX 线程库）：代码里要写 `#include <pthread.h>`；编译时传统上还要加 `-pthread` 选项（本项目的编译命令见 5.1 节）。本程序只用到一个类型和两个函数：

| 名字 | 作用 |
| --- | --- |
| `pthread_t` | 线程的编号牌。创建线程时由系统填写，等待线程时要用到 |
| `pthread_create(&tid, NULL, 函数, 参数)` | 创建一个新线程去执行"函数(参数)"，新线程的编号写进 `tid`。调用者**不会等**新线程，而是立即往下执行 |
| `pthread_join(tid, NULL)` | 等待编号为 `tid` 的线程结束。在那之前，调用者停在这一行不动，这叫**阻塞** |

交给 `pthread_create` 的函数必须写成 `void* 函数名(void* 参数名)` 的样子，只能接收一个 `void*` 参数（`void*` 是"可以指向任意类型数据的指针"）。要告诉线程好几个数时，就把它们装进一个结构体，把结构体的地址传过去；线程函数拿到后，再把它转换回结构体指针来读。

## 2.4 新问题：线程之间的先后顺序

Part I 的 11 个线程互不依赖，可以同时开工，谁先完成都行。Part II 不一样：**合并线程要用到两个排序线程的结果**。如果合并线程在两半还没排好时就开始合并，它读到的是没排好、甚至正在被修改的数据，结果必然出错。

这种"B 必须等 A 完成才能开始"的关系叫做**依赖**。让多个线程按正确的先后顺序执行，叫做线程的**同步**（synchronization），也就是实验目标里说的"协调线程的运行"。

本程序用最简单的办法保证顺序：

1. 主线程创建两个排序线程；
2. 主线程用 `pthread_join` 等这两个线程**都结束**；
3. 这时才创建合并线程。

在两半排好之前，合并线程根本还不存在，自然不可能提前开工。

> 了解即可：也可以一开始就把三个线程都创建出来，让合并线程自己调用 `pthread_join` 等两个排序线程，或者使用互斥锁、条件变量等专门的同步工具。本程序的写法最直接，用不到这些工具。

## 2.5 两个线程同时修改同一个数组，为什么不需要锁

如果两个线程同时访问**同一个位置**，并且至少有一个在写，结果就可能出错，这叫**数据竞争**（data race）。常见的解决办法是加锁，保证同一时刻只有一个线程能访问。

两个排序线程确实在同时修改同一个数组 `list`，但它们负责的是**互不重叠的两段**：线程 0 只读写前一半，线程 1 只读写后一半，从来不会碰到同一个位置，所以不会发生数据竞争，也不需要加锁。4.6 节

会看到代码是怎样保证"不越界"的。

其他共享数据也都是安全的：

| 共享数据 | 谁在什么时候访问 |
| --- | --- |
| 指针 `list`、`sorted` 本身，以及 `count` | 主线程在创建工作线程之前就设好了，之后所有线程只读不写 |
| `list` 的前一半 | 线程 0 排序时读写；线程 0 结束后，合并线程和主线程才去读 |
| `list` 的后一半 | 线程 1 排序时读写；线程 1 结束后，合并线程和主线程才去读 |
| `sorted` | 只有合并线程写；合并线程结束后，主线程才去读 |

POSIX 标准还保证：调用 `pthread_create` 之前写好的数据，新线程一定能看到；线程结束前写入的数据，`pthread_join` 返回后调用者一定能看到。所以排序线程排好的结果，合并线程一定能看到；合并线程的结果，主线程也一定能看到。

## 3. 程序全貌

### 3.1 一个比喻

把列表想象成桌上排成一行、写着数字的卡片：

| 程序里的东西 | 比喻 |
| --- | --- |
| 进程 | 一间教室 |
| 主线程（执行 `main`） | 组长 |
| 线程 0、线程 1（排序线程） | 两个组员，各自整理一半卡片 |
| 线程 2（合并线程） | 第三个组员，把整理好的两半合成一行 |
| 全局数组 `list` | 桌上排成一行的卡片，所有人都能看到、都能动 |
| 全局数组 `sorted` | 旁边另一行空位，用来摆放合并后的卡片 |
| `parameters` 结构体 | 组长发的任务纸条，写着"你负责从哪张到哪张" |
| `pthread_create` | 组长叫一个组员开工 |
| `pthread_join` | 组长等某个组员干完活 |
| `free(data)` | 组员看完纸条后把它扔掉 |

关键在于组长的安排：**等两个整理卡片的组员都说"好了"，才叫第三个组员来合并**。

## 3.2 代码由哪些部分组成

按在文件中出现的顺序：

| 代码 | 作用 |
| --- | --- |
| `#include`、`#define` | 引入库函数；定义常量 `NUM_THREADS`（3） |
| `parameters` 结构体 | 传给线程的参数：负责范围的起点 `start` 和终点 `end` |
| `int* list` | 全局变量：读入的列表，两个排序线程各排其中一半 |
| `int* sorted` | 全局变量：第二个数组，合并线程把结果写在这里 |
| `int count` | 全局变量：列表里有多少个数 |
| `sorter()` | 线程 0、1 执行的函数：用插入排序排好一段 |
| `merger()` | 线程 2 执行的函数：把两半合并进 `sorted` |
| `create_worker()` | 准备参数并创建一个线程（和 `sudoku.c` 里的同名函数几乎一样） |
| `read_list()` | 从标准输入读入所有整数 |
| `print_list()` | 打印数组中的一段 |
| `main()` | 主线程：读入 → 排序 → 合并 → 打印 |

C 语言要求函数先定义后使用，所以 `main` 写在文件最后；但程序是**从 `main` 开始执行**的。第 4 节按执行顺序讲解。

## 3.3 运行流程

```
main thread                          worker threads
──────────                           ──────────────
1. read the integers into list
2. allocate sorted, print the original list

3. create worker 0  ──────────────►  worker 0: sort the first half of list
   create worker 1  ──────────────►  worker 1: sort the second half of list
   join worker 0                     (workers 0 and 1 run at the same time)
   join worker 1

4. create worker 2  ──────────────►  worker 2: merge both halves into sorted
   join worker 2

5. print the results, free the memory
```

左边是主线程的 5 步，右边是工作线程。图中 create 对应 `pthread_create`，join 对应 `pthread_join`。

和 Part I 最大的不同在第 3、4 步之间：主线程先**等两个排序线程都结束**，才创建合并线程。所以程序分成两个阶段：先是两个排序线程同时工作，然后合并线程单独工作。

## 4. 按执行顺序读代码

### 4.1 开头：注释、头文件和常量

文件开头的英文注释大意是：多线程排序程序。从标准输入读入一串用空白分隔的整数，一直读到输入结束，然后用 3 个工作线程排序：线程 0 排前一半，线程 1 排后一半，线程 2 把排好的两半合并到第二个数组。主线程先打印原始列表，等各线程完成后，再打印排好的两半和最终的有序列表。最后是编译命令和两种运行方法（见 5.2）。

接着是：

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define NUM_THREADS 3
```

| 头文件 | 本程序用到的函数 |
| --- | --- |
| pthread.h | pthread_create、pthread_join |
| stdio.h | printf、scanf、fprintf、perror |
| stdlib.h | malloc、realloc、free、exit |
| string.h | strerror |

`NUM_THREADS` 是工作线程的个数：2 个排序线程加 1 个合并线程。

### 4.2 共享数据：结构体和全局变量

```
/* structure for passing data to threads */
typedef struct {
    int start; /* index of the first element */
    int end;   /* index one past the last element */
} parameters;

int* list;   /* the input list; each sorting thread sorts one half of it */
int* sorted; /* the merging thread merges the two sorted halves into here */
int count;   /* number of integers in list (and in sorted) */
```

注释的意思依次是：用来给线程传递数据的结构体；`start` 是第一个元素的下标；`end` 是最后一个元素的**下一个**下标；`list` 是读入的列表，每个排序线程排其中一半；`sorted` 用来存放合并线程合并好的结果；`count` 是 `list`（也是 `sorted`）中整数的个数。

`parameters`：**告诉线程"你负责哪一段"**。实验文档要求告诉每个排序线程"从哪个下标开始排序"。本程序传两个数：起点 `start` 和终点 `end`。

这个范围是**左闭右开**的，和数学里的区间 [a, b) 一样：包含 `start`，不包含 `end`。代码注释里写的 `list[start, end)` 是一种简写，不是 C 语法，意思是"`list` 中下标从 `start` 到 `end - 1` 的那一段"。以 10 个数为例：

| 下标 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 原始值 | 7 | 12 | 19 | 3 | 18 | 4 | 2 | 6 | 15 | 8 |
| 负责的线程 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 |

线程 0 收到 `start = 0`、`end = 5`，负责下标 0～4；线程 1 收到 `start = 5`、`end = 10`，负责下标 5～9。

这样写有三个好处：

- 元素个数就是 `end - start`；
- 前一半的 `end` 正好等于后一半的 `start`（都是 5），两段首尾相接，既不重叠，也不遗漏；
- 循环可以直接写成 `for (i = start; i < end; i++)`。

三**个全局变量**定义在所有函数外面，所有线程都能访问（见 2.2）。

注意 `list` 和 `sorted` 是**指针**，而不是像 `sudoku.c` 里 `int sudoku[SIZE][SIZE]` 那样写死大小的数组。原因是：程序运行之前不知道输入会有多少个数，所以只能先声明指针，读入时再用 `realloc`、`malloc` 申请大小合适的内存（见 4.3、4.4），让指针指向这块内存。之后就可以像普通数组一样，用 `list[i]` 访问第 i 个数。

全局变量在程序启动时会自动初始化为 0，对指针来说就是 `NULL`（空指针，表示"还没有指向任何内存"）。

> **C 语法：把指针当数组用**
> 如果指针 `p` 指向一块能放 n 个 `int` 的内存，那么 `p[0]` 到 `p[n - 1]` 就是这块内存里的 n 个 `int`，用法和数组完全一样。

## 4.3 main 第 1 步：读入列表

`main` 先声明了两个局部变量：

```
    pthread_t workers[NUM_THREADS];
    int middle;
```

`workers` 是 3 个线程的编号牌，`middle` 是列表分成两半的位置（见 4.5）。`main` 的第一行代码是 `read_list();`，读入整个列表：

```
/* Read integers from standard input until end of file into list, enlarging
 * list as needed, and set count. */
void read_list(void) {
    int capacity = 0;
    int value;
    int status;

    while ((status = scanf("%d", &value)) == 1) {
        if (count == capacity) {
            capacity = (capacity == 0) ? 16 : capacity * 2;
            list = (int*)realloc(list, capacity * sizeof(int));
            if (list == NULL) {
                perror("realloc");
                exit(EXIT_FAILURE);
            }
        }
        list[count++] = value;
    }

    if (status != EOF || count == 0) {
        fprintf(stderr,
                "Error: expected integers separated by whitespace on standard "
                "input\n");
        exit(EXIT_FAILURE);
    }
}
```

注释的意思是：从标准输入读整数，一直读到输入结束，存进 `list`；空间不够时扩大 `list`；同时设置好 `count`。

**读一个数。** `scanf("%d", &value)` 从标准输入读一个整数存进 `value`。它的返回值有三种情况：

| 返回值 | 含义 |
|---|---|
| 1 | 成功读到一个整数 |
| 0 | 接下来的内容不是整数（例如逗号、字母），读取失败 |
| EOF | 输入已经结束（EOF 是 stdio.h 定义的一个特殊值，通常是 -1） |

`while ((status = scanf("%d", &value)) == 1)` 先把返回值存进 `status`，再判断它是否等于 1：只要还能读到整数，循环就继续。

**存一个数。** `list[count++] = value;` 把 `value` 存进 `list[count]`，然后 `count` 加 1。所以循环结束时，`count` 正好是读到的整数个数。

**空间不够时扩大。** 程序事先不知道总共有多少个数，所以一边读一边扩大 `list`。`capacity` 是 `list` 当前最多能放几个数，`count` 是已经放了几个数。两者相等，说明放满了，需要扩大：

- 第一次（`capacity` 为 0）先申请能放 16 个数的空间，以后每次放满就扩大到原来的 2 倍：16 → 32 → 64 → ……
- 每次扩大都可能要把已有的数整体搬到新位置。按 2 倍扩大，而不是每次只多申请 1 个，扩大的次数就少得多。

就像记账的本子写满了，就换一本两倍厚的，把旧内容抄过去，再接着写。

**检查读入是否正常。** 循环结束有两种可能：输入正常结束（`status` 等于 `EOF`），或者遇到了不是整数的内容（`status` 等于 0）。如果是后者，或者一个数都没读到（`count == 0`），就向 `stderr`（专门输出错误信息的通道）打印错误提示，再用 `exit(EXIT_FAILURE)` 结束整个程序，退出码为 1。

> **C 语法：`realloc`**
>
> `realloc(p, n)` 把 `p` 指向的内存块改成 n 个字节，原来的内容保持不变，返回新内存块的地址。内存块可能被搬到了别处，所以要用返回值更新 `list`。如果 `p` 是 `NULL`，`realloc` 的效果和 `malloc(n)` 一样：第一次调用时 `list` 还是 `NULL`，正好用上这一点。申请失败时返回 `NULL`。`capacity * sizeof(int)` 是放 `capacity` 个 `int` 需要的字节数。

> **C 语法：本段用到的其他写法**
>
> - `(status = scanf(...)) == 1`：在 C 语言里，赋值本身也有值，就是被赋的那个值。外面那层括号不能省：没有它，会先算 `scanf(...) == 1`，再把比较的结果（0 或 1）赋给 `status`。
> - `count++`：先用 `count` 原来的值，再把它加 1。`list[count++] = value;` 等价于 `list[count] = value;` 加上 `count = count + 1;` 两句。
> - `条件 ? A : B`：条件为真取 A，否则取 B。`(capacity == 0) ? 16 : capacity * 2` 就是"`capacity` 为 0 时取 16，否则取它的 2 倍"。

这一步只有主线程在运行，工作线程还不存在，所以修改 `list` 和 `count` 不会和任何线程冲突。

### 4.4 main 第 2 步：准备第二个数组，打印原始列表

```
    sorted = (int*)malloc(count * sizeof(int));
    if (sorted == NULL) {
        perror("malloc");
        return EXIT_FAILURE;
    }
    print_list("Original list:", list, 0, count);
```

- 读完列表后 `count` 已经确定，于是用 `malloc` 申请一块能放 `count` 个 `int` 的内存，让 `sorted` 指向它。这就是实验文档里说的"和原数组一样大的第二个全局数组"，合并线程会把结果写在这里。
- 然后打印原始列表。这一步必须在排序**之前**做：排序线程直接在 `list` 上排序（称为**原地排序**），排完之后原来的顺序就没有了。

打印用的 `print_list`：

```
/* Print label followed by array[start, end) on one line. */
void print_list(const char* label, const int* array, int start, int end) {
    int i;

    printf("%-17s", label);
    for (i = start; i < end; i++) printf(" %d", array[i]);
    printf("\n");
}
```

注释的意思是：先打印标题 `label`，再把 `array` 中下标 `start` 到 `end - 1` 的数打印在同一行。

- `%-17s`：标题至少占 17 个字符宽，靠左对齐，右边补空格。17 正好是最长的标题 `Sorting thread 0:` 的长度，这样各行的数字都能对齐。
- `" %d"`：每个数前面加一个空格。
- `array` 既可以是 `list`，也可以是 `sorted`，同一个函数能打印两种数组。

> **C 语法：`const`**
>
> `const char* label` 表示"这个函数不会修改 `label` 指向的字符"，`const int* array` 同理。`const` 是给编译器和读代码的人的承诺：这个函数只读不写。

### 4.5 main 第 3 步：创建两个排序线程，并等它们结束

```
    /* split the list in two; if count is odd, the second half is one longer */
    middle = count / 2;
    create_worker(&workers[0], sorter, 0, middle);
    create_worker(&workers[1], sorter, middle, count);
    pthread_join(workers[0], NULL);
    pthread_join(workers[1], NULL);
```

注释的意思是：把列表分成两半；如果 `count` 是奇数，后一半多一个数。

**分成两半。** C 语言里两个整数相除只保留整数部分（`5 / 2` 等于 2），所以：

| count | middle = count / 2 | 线程 0 负责 | 线程 1 负责 |
|---|---|---|---|
| 10 | 5 | 下标 0～4，共 5 个数 | 下标 5～9，共 5 个数 |
| 5 | 2 | 下标 0～1，共 2 个数 | 下标 2～4，共 3 个数 |
| 1 | 0 | 没有数 | 下标 0，共 1 个数 |

实验文档说"分成大小相等的两个列表"，但个数为奇数时不可能完全相等，本程序让后一半多一个。

**创建两个排序线程。** 两个线程执行的是同一个函数 `sorter`，区别只在纸条上写的范围：线程 0 是 `[0, middle)`，线程 1 是 `[middle, count)`。

`create_worker` 和 `sudoku.c` 里的同名函数几乎一样，只是纸条上写的是 `start` 和 `end`：

```
void create_worker(pthread_t* tid, void* (*routine)(void*), int start,
                   int end) {
    parameters* data = (parameters*)malloc(sizeof(parameters));
    int err;

    if (data == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }
    data->start = start;
    data->end = end;

    err = pthread_create(tid, NULL, routine, data);
    if (err != 0) {
        fprintf(stderr, "pthread_create: %s\n", strerror(err));
        exit(EXIT_FAILURE);
    }
}
```

它做了三件事：

1. **申请一张纸条**：用 `malloc` 申请一个 `parameters` 结构体大小的内存。申请失败时，用 `perror` 打印原因，再用 `exit` 结束程序。
2. **在纸条上写范围**：`data->start = start;` 和 `data->end = end;`。`data->start` 表示"`data` 指向的结构体的 `start` 成员"。
3. **创建线程**：`pthread_create(tid, NULL, routine, data)` 创建新线程，让它执行 `routine(data)`，并把新线程的编号写进 `*tid`（也就是 `main` 里的 `workers[i]`）。失败时用 `strerror` 把错误码翻译成文字打印出来，然后结束程序。

第二个参数 `void* (*routine)(void*)` 是**函数指针**：它指向一个"参数为 `void*`、返回值为 `void*`"的函数，也就是线程函数。所以 `create_worker(&workers[0], sorter, 0, middle)` 的意思就是"创建一个新线程去执行 `sorter`，负责 `[0, middle)`，编号存进 `workers[0]`"。纸条为什么必须用 `malloc` 申请、而不能用局部变量，GUIDE-1 第 6 节问题 1 有详细解释。

**等两个排序线程结束。** 两次 `create_worker` 之后，两个排序线程已经在同时工作了。主线程接着执行 `pthread_join(workers[0], NULL)`，停在这里等线程 0 结束，然后执行 `pthread_join(workers[1], NULL)` 等线程 1。如果线程 1 比线程 0 先结束也没关系，第二个 `pthread_join` 会立刻返回。两行都执行完时，两半一定都已排好。

### 4.6 排序线程在做什么：`sorter`

```c
/* Sorting thread: sort list[start, end) in place with insertion sort. */
void* sorter(void* param) {
    parameters* data = (parameters*)param;
    int i, j;

    for (i = data->start + 1; i < data->end; i++) {
        int value = list[i];

        for (j = i - 1; j >= data->start && list[j] > value; j--)
            list[j + 1] = list[j];
        list[j + 1] = value;
    }

    free(data);
    return NULL;
}
```

注释的意思是：排序线程，用插入排序把 `list[start, end)` 原地排好序。

外层的框架和 `sudoku.c` 里的线程函数一样：

1. `parameters* data = (parameters*)param;`：把收到的 `void*` 转回 `parameters*`，才能读纸条上的 `data->start` 和 `data->end`；
2. 中间的两层 `for` 循环完成排序；

3. `free(data);`：纸条用完了，归还内存；
4. `return NULL;`：函数返回，这个线程就结束了。

排序的结果不需要专门"交回"给主线程：线程直接在全局数组 `list` 上排序，排完后，主线程和合并线程都能看到。

**插入排序**和打牌时整理手里的牌一样：左手的牌已经排好序，每次从右边摸一张新牌，从右往左找到它该在的位置，把比它大的牌都往右挪一格，再把新牌插进空出来的位置。对应到代码：

- 外层循环的 `i` 是"新摸的牌"的位置，`i` 左边的那一段已经排好序。一开始左边只有 `list[start]` 一张牌，它自己就是有序的，所以 `i` 从 `start + 1` 开始。
- `value = list[i]` 把新牌拿在手里。
- 内层循环让 `j` 从 `i - 1` 往左走：只要 `list[j]` 比 `value` 大，就把它右移一格（`list[j + 1] = list[j]`）。
- 内层循环停下时，要么 `list[j]` 不比 `value` 大，要么 `j` 已经走出了范围的左端。不管哪种情况，`value` 都该放在 `j` 的右边：`list[j + 1] = value`。

以线程 0 负责的前一半 `7 12 19 3 18` 为例：

| 步骤 | 拿出的数 `value` | 处理后的前一半 | 说明 |
|---|---|---|---|
| 开始 | | 7 12 19 3 18 | 只看第一个数 7，它自己就是有序的 |
| `i = 1` | 12 | 7 12 19 3 18 | 7 不比 12 大，12 留在原处 |
| `i = 2` | 19 | 7 12 19 3 18 | 12 不比 19 大，19 留在原处 |
| `i = 3` | 3 | 3 7 12 19 18 | 19、12、7 都比 3 大，依次右移一格，3 放到最前面 |
| `i = 4` | 18 | 3 7 12 18 19 | 19 比 18 大，右移一格；12 不比 18 大，18 放在 12 后面 |

把 `i = 3` 这一步拆开，每执行一句赋值，前一半变成：

```
value = 3          7 12 19  3 18
list[3] = list[2]  7 12 19 19 18
list[2] = list[1]  7 12 12 19 18
list[1] = list[0]  7  7 12 19 18
list[0] = value    3  7 12 19 18
```

"右移"其实是复制：执行 `list[3] = list[2]` 之后，19 同时出现在下标 2 和 3，下标 2 上的那个会在下一句被覆盖。被拿出来的 3 已经保存在 `value` 里，不会丢。

线程 1 用同样的方法，把后一半 `4 2 6 15 8` 排成 `2 4 6 8 15`。

**为什么线程 0 和线程 1 不会互相干扰？** 关键是内层循环的条件 `j ≥ data->start`：`j` 往左最多走到自己范围的起点，不会越过去；外层循环的条件 `i < data->end` 保证不会越过终点。所以线程 1（范围 `[5,`

`10)`）绝不会碰到下标 0～4，线程 0 也绝不会碰到下标 5～9。另外，`i`、`j`、`value` 都是局部变量，每个线程各有一份（见 2.2）。

> **C 语法：`&&` 的"短路"**
>
> `A && B` 从左往右计算，如果 `A` 为假，就不再计算 `B`。在 `j ≥ data→start && list[j] > value` 中，`j` 一旦走出范围，左边就为假，程序不会再去读 `list[j]`。对线程 0 来说，这避免了读 `list[-1]`（数组越界）；对线程 1 来说，这避免了去读线程 0 正在修改的那一半。

## 4.7 main 第 4 步：创建合并线程，并等它结束

```
    /* the merging thread may start only after both halves are sorted */
    create_worker(&workers[2], merger, 0, count);
    pthread_join(workers[2], NULL);
```

注释的意思是：两半都排好之后，合并线程才可以开始。

执行到这里时，上一步的两个 `pthread_join` 都已经返回，两半一定都排好了。主线程这才创建合并线程，纸条上写的范围是整个列表 `[0, count)`，然后等它结束。

## 4.8 合并线程在做什么：`merger`

```
/* Merging thread: list[start, end) was split at its middle and each half was
 * sorted by a sorting thread; merge the two halves into sorted[start, end). */
void* merger(void* param) {
    parameters* data = (parameters*)param;
    int middle = (data->start + data->end) / 2;
    int i = data->start; /* next element of the first half */
    int j = middle;      /* next element of the second half */
    int k = data->start; /* next free position in sorted */

    while (i < middle && j < data->end) {
        if (list[i] ≤ list[j])
            sorted[k++] = list[i++];
        else
            sorted[k++] = list[j++];
    }
    while (i < middle) sorted[k++] = list[i++];
    while (j < data->end) sorted[k++] = list[j++];

    free(data);
    return NULL;
}
```

注释的意思是：合并线程。`list[start, end)` 从中间分成了两半，每一半都已由一个排序线程排好；把这两半合并进 `sorted[start, end)`。三个变量旁边的注释分别是：前一半里下一个要处理的数、后一半里下一个要处理的数、`sorted` 里下一个空位。

**找到两半的分界。** 合并线程收到的范围是 `[0, count)`，它用 `(data->start + data->end) / 2`，也就是 `(0 + count) / 2` 算出中点。这和 `main` 里的 `middle = count / 2` 是同一个数，所以合并线程找到的分界，正好就是两个排序线程的分界。

**三根"手指"。** `i` 指着前一半里下一个还没用过的数，`j` 指着后一半里下一个还没用过的数，`k` 指着 `sorted` 里下一个要填的空位。

合并就像把两叠已经排好序的卡片合成一叠：每次比较两叠最上面的两张，把较小的那张拿走，放到新的一叠上；某一叠拿完了，就把另一叠剩下的卡片按顺序全部放过去。对应到代码：

- 第一个 `while`：两半都还有数时，比较 `list[i]` 和 `list[j]`，把较小的放进 `sorted`，对应的手指往后移一格。两个数相等时取前一半的。
- 后两个 `while`：总有一半先用完。另一半剩下的数本来就有序，而且都不比已经放进去的数小，照原样抄到 `sorted` 末尾即可。这两个循环最多只有一个会真正执行。

以 `3 7 12 18 19` 和 `2 4 6 8 15` 为例：

| 步骤 | 比较 | 放进 sorted 的数 | 此时的 sorted |
|---|---|---|---|
| 1 | 3 和 2 | 2 | 2 |
| 2 | 3 和 4 | 3 | 2 3 |
| 3 | 7 和 4 | 4 | 2 3 4 |
| 4 | 7 和 6 | 6 | 2 3 4 6 |
| 5 | 7 和 8 | 7 | 2 3 4 6 7 |
| 6 | 12 和 8 | 8 | 2 3 4 6 7 8 |
| 7 | 12 和 15 | 12 | 2 3 4 6 7 8 12 |
| 8 | 18 和 15 | 15 | 2 3 4 6 7 8 12 15 |
| 9 | 后一半已用完 | 18 | 2 3 4 6 7 8 12 15 18 |
| 10 | 后一半已用完 | 19 | 2 3 4 6 7 8 12 15 18 19 |

前 8 步发生在第一个 `while` 里。第 8 步取走 15 之后，后一半用完了，第 9、10 步由第二个 `while` 把前一半剩下的 18、19 抄过去。

合并线程运行时，两个排序线程已经结束，所以它独自读 `list`、写 `sorted`，不会和任何线程冲突。

> **C 语法：** `sorted[k++] = list[i++];`
>
> 和 4.3 节的 `count++` 一样，`k++` 、`i++` 都是"先用原来的值，再加 1"。这一句等价于三句：`sorted[k] = list[i];` 、`k = k + 1;` 、`i = i + 1;` 。

### 4.9 main 第 5 步：打印结果，释放内存

```
    print_list("Sorting thread 0:", list, 0, middle);
    print_list("Sorting thread 1:", list, middle, count);
    print_list("Sorted list:", sorted, 0, count);

    free(list);
    free(sorted);
    return 0;
```

- 前两行打印两个排序线程各自排好的一半。合并线程只读 `list`、不修改它，所以这时 `list` 里还是两个排序线程的结果。
- 第三行打印合并后的完整有序列表 `sorted`，这正是实验文档要求父线程输出的结果。
- `free(list)` 和 `free(sorted)` 归还用 `realloc`、`malloc` 申请的内存。程序结束时操作系统也会回收进程的全部内存，但"申请的内存用完就归还"是好习惯。
- `main` 返回 0，表示程序正常结束。

这些打印都发生在三个工作线程全部结束之后，所以输出的内容和顺序每次都一样。

## 5. 编译、运行和看懂输出

### 5.1 编译

```
mkdir -p build && gcc -std=c99 -O2 -Wall -fno-asm "sort.c" -o "build/sort" --static -lm
```

这条命令先生成 `build` 目录，再编译。各部分的含义：

| 部分 | 含义 |
| --- | --- |
| `mkdir -p build` | 创建 build 目录；-p 表示目录已存在时不报错 |
| `&&` | 前一条命令成功后才执行后一条 |
| `gcc` | C 语言编译器 |
| `-std=c99` | 按 C99 标准编译 |
| `-O2` | 开启二级优化，让程序运行得更快 |

| 部分 | 含义 |
| --- | --- |
| -Wall | 打开常用的警告，帮你发现可疑的代码 |
| -fno-asm | 禁用 asm 关键字（本程序没有使用内联汇编） |
| "sort.c" | 源代码文件；引号可以省略，写成 sort.c 效果一样 |
| -o "build/sort" | 把可执行文件输出到 build/sort |
| --static | 静态链接，生成的可执行文件不依赖系统的动态库 |
| -lm | 链接数学库（本程序没有直接用到，保留在命令中不影响结果） |

没有任何输出就说明编译成功，`build` 目录下会多出一个 `sort` 文件，运行时写 `./build/sort`。

## 5.2 三种输入方式

程序会一直读到**输入结束**才开始排序。下面三种方式都可以：

**方式一：用 `echo` 和管道。**

```
echo 7 12 19 3 18 4 2 6 15 8 | ./build/sort
```

`echo` 把后面的内容原样输出；`|` 叫**管道**，把左边命令的输出接到右边程序的标准输入。

**方式二：从文件读。** 新建一个文本文件 `list.txt`，数与数之间用空格或换行隔开都可以：

```
7 12 19 3 18
4 2 6 15 8
```

然后运行：

```
./build/sort < list.txt
```

`<` 叫**输入重定向**，把文件的内容当作程序的标准输入。

**方式三：用键盘输入。** 只运行 `./build/sort`，然后输入数字（可以分多行）。输完后程序看起来像"卡住"了，其实它还在等更多的数。换到新的一行，按 **Ctrl+D**，告诉程序"输入结束了"，它才会开始排序。

注意：运行时一定要写 `./build/sort`，不能只写 `sort`，原因见第 6 节问题 6。

## 5.3 看懂输出

用实验文档里的列表运行，输出：

```
Original list:    7 12 19 3 18 4 2 6 15 8
Sorting thread 0: 3 7 12 18 19
Sorting thread 1: 2 4 6 8 15
Sorted list:      2 3 4 6 7 8 12 15 18 19
```

| 输出行 | 含义 |
| --- | --- |
| `Original list:` | 读入的原始列表 |
| `Sorting thread 0:` | 线程 0 排好的前一半 |
| `Sorting thread 1:` | 线程 1 排好的后一半 |
| `Sorted list:` | 线程 2 合并出的完整有序列表 |

第一行在创建线程之前打印，后三行在三个工作线程全部结束之后打印。结果和实验文档的 Fig. 2 完全一致。

## 5.4 动手试试

**奇数个数：**

```
echo 5 3 1 4 2 | ./build/sort
```

```
Original list:    5 3 1 4 2
Sorting thread 0: 3 5
Sorting thread 1: 1 2 4
Sorted list:      1 2 3 4 5
```

`count` 为 5，`middle` 为 2：线程 0 排前 2 个数，线程 1 排后 3 个数（见 4.5 节的表）。

**只有一个数：**

```
echo 42 | ./build/sort
```

```
Original list:    42
Sorting thread 0:
Sorting thread 1: 42
Sorted list:      42
```

`middle` 为 0，前一半是空的，所以 `Sorting thread 0:` 后面什么也没有。线程 0 照样被创建、照样运行，只是没有数可排，`sorter` 的外层循环一次也不执行。

**错误的输入：** 实验文档的图里用逗号分隔数字，但本程序只接受用空格或换行分隔：

```
echo 7, 12, 19 | ./build/sort
echo $?
```

```
Error: expected integers separated by whitespace on standard input
1
```

`scanf` 读完 7 之后遇到逗号，返回 0，`read_list` 于是报错，程序以退出码 1 结束。`echo $?` 打印上一条命令的**退出码**：0 表示正常结束，非 0 表示出错。

## 6. 常见疑问

**问题 1：为什么要等两个排序线程都结束，才创建合并线程？三个线程一起创建不行吗？**

如果三个线程一起创建、又不做任何同步，合并线程可能在两半还没排好时就开始合并。它读到的是部分有序、甚至正在被排序线程修改的数据，合并结果就会出错。

更麻烦的是，线程的执行顺序每次都可能不同，这样的程序可能这次运行正确、下次运行出错，很难发现和调试。所以必须用 `pthread_join` 这样的同步手段**明确保证**先后顺序，而不能指望运气（见 2.4）。

**问题 2：两个排序线程谁先排完？会影响结果吗？**

谁先排完不一定，每次运行都可能不同，由操作系统的调度决定。但这不影响结果：两个线程负责的是互不重叠的两半，谁先谁后都一样；合并线程又是在两个排序线程都结束之后才创建的。

**问题 3：为什么合并结果要写进另一个数组 `sorted`，不能直接写回 `list`？**

因为直接写回会覆盖还没用到的数。比如前一半是 3 7，后一半是 2 4：合并的第一步是把最小的 2 写到下标 0。如果写回 `list`，下标 0 上原来的 3 就被覆盖了，而 3 还没有被合并进去。写进另一个数组 `sorted`，`list` 里的数据就始终完好。这就是实验文档中"第二个全局数组"的用处。

**问题 4：用了多线程，排序就更快了吗？**

数很少时不会：创建线程的开销远大于排 10 个数的开销。数很多时会，原因有两个：

1. 在多核 CPU 上，两个排序线程真正同时运行，花的时间接近只排一半所需的时间。
2. 插入排序的耗时大约和数的个数的**平方**成正比。两半各有 $n/2$ 个数，分别排序的总工作量约为 $2 \times (n/2)^2 = n^2/2$，只有直接排整个列表（约 $n^2$）的一半。合并只需要把每个数过一遍，花的时间相对很少。

可以用 10 万个随机数做个实验，看看第 1 个原因的效果：

```
shuf -i 1-1000000 -n 100000 > big.txt
time ./build/sort < big.txt > /dev/null
time taskset -c 0 ./build/sort < big.txt > /dev/null
```

- `shuf -i 1-1000000 -n 100000`：从 1 到 1000000 中随机选 10 万个不同的数，每行一个。
- `> /dev/null`：把输出丢掉，免得刷屏。
- `time`：命令结束后报告用时。`real` 是实际经过的时间；`user` 是所有线程在 CPU 上执行程序代码的时间之和。
- `taskset -c 0`：让程序只能在 0 号 CPU 核心上运行，两个排序线程只能轮流执行。

下面是在一台多核 Linux 机器上测得的结果。你的数字会不同，但只要机器有 2 个或更多核心，规律应该一样：

| 运行方式 | real | user |
| --- | --- | --- |
| 不限制核心 | 约 0.17 秒 | 约 0.32 秒 |
| 只用 1 个核心 | 约 0.32 秒 | 约 0.32 秒 |

只用 1 个核心时，用时差不多翻了一倍。不限制核心时，`user` 大约是 `real` 的两倍，说明有两个核心在同时干活。

**问题 5：为什么选插入排序？可以换成别的排序算法吗？**

实验文档允许使用任意排序算法。插入排序代码短、容易看懂，而且直接在原数组上排序，正好符合"每个排序线程处理数组的一半"的设计。它的缺点是数多了以后很慢：耗时和个数的平方成正比，10 万个数约 0.17 秒，100 万个数（10 倍）要约 16 秒（约 100 倍）。

如果要换算法（例如归并排序、快速排序，或者 C 标准库的 `qsort`），只需要改写 `sorter` 中负责排序的部分，线程的创建、传参和合并都不用改。

**问题 6：为什么运行时要写 `./build/sort`，不能只写 `sort`？**

在终端里输入一个不带路径的命令名时，系统会到一些固定的目录（由环境变量 `PATH` 指定，例如 `/usr/bin`）里去找这个程序，而那里正好有一个系统自带的 `sort` 命令。它按文本逐行排序，和本程序是两回事。`./build/sort` 是一个相对路径：`./` 表示当前目录，所以它明确指向"当前目录下 `build` 子目录里的 `sort`"，运行的才是你刚编译出来的程序。

# 7. 小结

| 实验要求 | 代码中的实现 |
| --- | --- |
| 列表分成两个大小相等的子列表 | `middle = count / 2`；个数为奇数时后一半多 1 个 |
| 两个排序线程各排一半，算法自选 | 线程 0、1 执行 `sorter`，用插入排序在 `list` 上原地排序 |
| 第三个线程把两半合并进第二个全局数组 | 线程 2 执行 `merger`，把结果写进 `sorted` |
| 告诉每个排序线程从哪里开始排 | `parameters` 结构体（`start`、`end`），`malloc` 后交给 `pthread_create`，线程用完后 `free` |
| 所有线程结束后由父线程输出 | 主线程用 `pthread_join` 等两个排序线程，再创建并等待合并线程，最后打印 |

和 Part I 的 `sudoku.c` 对比：

| | `sudoku.c` | `sort.c` |
| --- | --- | --- |
| 工作线程 | 11 个，一起创建、同时工作 | 3 个，分两批：先 2 个排序线程，再 1 个合并线程 |
| 线程之间的关系 | 互不依赖 | 合并线程依赖两个排序线程的结果 |
| 共享数据 | 大小固定的全局数组 | 全局指针，指向运行时按输入大小申请的内存 |
| 结果怎样交出去 | 每个线程写 `result` 中自己的那一格 | 排序线程直接排好 `list` 中自己的那一半；合并线程写 `sorted` |
| 为什么不需要锁 | 每个线程只写自己的那一格 | 每个线程只碰自己的那一段；有依赖的线程靠 `pthread_join` 排好先后 |